

Giảng viên ra đề: (Chữ ký và Họ tên)	(Ngày ra đề)	Người phê duyệt: (Chữ ký và họ tên)	(Ngày duyệt đề)
--	--------------	---	-----------------

 TRƯỜNG ĐH BÁCH KHOA - ĐHQG-HCM KHOA KH & KT MÁY TÍNH	THI CUỐI KỲ		Học kỳ / Năm học		1	2022-2023
			Ngày thi		21-12-2022	
	Môn học	Cấu trúc dữ liệu và giải thuật				
	Mã môn học	CO2003				
	Thời lượng	80 phút	Mã đề	2211		
Ghi chú: - Sinh viên không sử dụng tài liệu. - Sinh viên làm trực tiếp trên đề và nộp lại. - Quy ước chọn ưu tiên trong phép duyệt là thứ tự alphabet. - Quy ước so sánh trên cây nhị phân dùng ký tự là theo thứ tự alphabet.						

I. Trắc nghiệm (60p)

1. [3.1M] Duyệt trung thứ tự cây nhị phân cho kết quả là (A,B,C,D,E,F,G), duyệt hậu thứ tự cho kết quả là (B,D,C,A,F,G,E), hãy cho biết cây con bên phải của nút gốc có tổng cộng bao nhiêu nút

- (a) 2
(b) 3
(c) 4
(d) 5

2. [3.1M] Lần lượt thêm các khoá 5; 7; 12; 25; 36; 58 vào một bảng băm có kích thước là 11, sử dụng hàm băm $h(k) = k \bmod 11$. Nếu đụng độ xảy ra thì phương pháp giải quyết đụng độ được chọn là mở rộng địa chỉ (open addressing) với hàm dò tìm là $p(k, i) = h(k) + 2 * i + 1$. Vị trí của khoá 58 trong bảng băm là:

- (a) 0
(b) 3
(c) 6
(d) 9

3. [2M] Cho cây AVL với 2 nút, trong đó nút gốc có giá trị 15, nút còn lại có giá trị 20. Khi chèn thêm nút có giá trị 19, cần phải

- (a) Thực hiện phép quay đơn (single rotation) về bên trái
(b) Thực hiện phép quay đơn về bên phải
(c) Thực hiện phép quay kép (double rotation)
(d) Không cần thực hiện phép quay

4. [3.1H] Xây dựng cây B (B-Tree) bậc 3 từ dãy khóa sau (18, 4, 2, 46, 48, 29, 30). Hãy cho biết chèn các khóa nào dưới đây vào cây B ở trên, thì sẽ xảy ra việc tách nút (split node)

- (a) 1, 32, 53
(b) 1, 19, 20
(c) 1, 19, 32
(d) 1, 19, 32, 53

5. [1M] Giải thuật sau tìm kiếm target trong mảng tăng dần a:

```

1  int i, low = 0, high = n - 1;
2  while (low < high) {
3      i = (low + high) / 2;
4      if (a[i] == target) return i;
5      else if (a[i] < target) low = i + 1;
6      else high = i - 1;
7  }
8  return -1;

```

Với $n = 13$ và $target > a[n-1]$, hãy cho biết số lần thực hiện phép so sánh trong các câu lệnh if

- (a) 4
(b) 6
(c) 7
(d) 8

6. [2.1E] Mảng nào sau đây biểu diễn một min-heap?

- (a) 1, 2, 8, 4, 13, 6, 9, 15
(b) 1, 2, 4, 13, 6, 8, 15, 9
(c) 1, 2, 13, 15, 6, 4, 9, 8
(d) 1, 2, 6, 4, 13, 8, 9, 15

7. [2.1M] Cho biết nhận định nào dưới đây về cây B (B-Tree) bậc m là KHÔNG đúng

- (a) Nút gốc có nhiều nhất m cây con
(b) Tất cả các nút lá nằm trên cùng một mức
(c) Các khóa trong nút gốc được sắp xếp theo thứ tự
(d) Nút trong có ít nhất $m/2 + 1$ cây con khác rỗng (nếu m chẵn) hoặc $m/2 - 1$ cây con khác rỗng (nếu m lẻ)

8. [3.1E] Phát biểu nào sau đây là đúng về Quadratic probing?
- Là một phương pháp hiện thực hàm băm.
 - Có thể dẫn đến Primary clustering.
 - Có thể dẫn đến Secondary clustering.
 - Có thể dẫn đến Linear clustering và Secondary clustering.
9. [2.1E] Phát biểu nào sau đây là đúng khi nói về một Node trong danh sách liên kết?
- Node có thể có nhiều trường dữ liệu.
 - Node phải có 1 trường dữ liệu thuộc kiểu số nguyên.
 - Node phải có 1 trường dữ liệu thuộc kiểu T.
 - Node có thể không có liên kết đến Node khác.
10. [2.1E] Biết rằng mảng A dùng để biểu diễn một max-heap và một chỉ số i. Biết rằng việc truy cập các phần tử trong mảng đều hợp lệ, biểu thức nào sau đây LUÔN ĐÚNG?
- $A[i] > A[2i + 1]$
 - $A[i] < A[2i + 2]$
 - $A[i] > A[2i]$
 - $A[i] < A[2i - 1]$
11. [2.1M] Lần lượt thêm các số sau vào 1 cây AVL rỗng: 3, 10, 20, 9, 1, 5. Liệt kê các Node nằm ở tầng 2 (level 2) trong cây cuối cùng thu được theo thứ tự tăng dần.
- 3, 5, 20
 - 3, 10
 - 1, 5, 20
 - 3, 5, 10
12. [1.2M] Cho đoạn chương trình sau:
- ```

1 int func(int x){
2 if (x == 0) return 1;
3 if (x == 1) return func(0);
4 return func(x - 1) + x;
5 }

```
- Số lần phép so sánh được thực thi khi gọi hàm f(N) (với  $N > 0$ ) là:
- N
  - 2N
  - $2N + 1$
  - $N^2$
13. [2E] Thông thường, để lưu trữ một ma trận số, người ta sử dụng một mảng 2 chiều kích thước  $m \times n$  để chứa các phần tử này. Giả sử, ta sử dụng một mảng một chiều kích thước N để chứa các phần tử của ma trận trên. Khẳng định nào sau đây là đúng khi so sánh 2 chương trình in mảng một chiều kích thước N và mảng 2 chiều kích thước  $m \times n$ ?
- Chương trình in mảng hai chiều sẽ nhanh hơn hẳn chương trình còn lại
  - Chương trình in mảng một chiều sẽ nhanh hơn hẳn do có độ phức tạp  $O(N)$
  - Hai chương trình có tốc độ gần như tương đương nhau
  - Các đáp án trên đều sai
14. [3.1H] Biết rằng duyệt tiền thứ tự một cây nhị phân cho kết quả là (A,B,C,D,E,K,F,G), kết quả duyệt trung thứ tự là (B,C,A,D,K,E,F,G). Hãy cho biết kết quả duyệt hậu thứ tự
- C,B,K,G,F,E,D,A
  - C,B,K,G,E,F,D,A
  - C,B,G,K,F,E,D,A
  - Cả ba đáp án còn lại đều sai
15. [3.1H] Cho cây AVL có kết quả duyệt tiền thứ tự là (13, 10, 5, 4, 6, 11, 15, 16), kết quả duyệt trung thứ tự là (4, 5, 6, 10, 11, 13, 15, 16). Chèn thêm nút có giá trị 7 vào cây AVL ở trên, kết quả duyệt tiền thứ tự sau khi chèn giá trị 7 là
- 13, 5, 4, 6, 10, 7, 11, 15, 16
  - 13, 6, 5, 4, 10, 7, 11, 15, 16
  - 6, 13, 5, 4, 10, 7, 11, 15, 16
  - 6, 5, 4, 10, 7, 11, 13, 15, 16
16. [2.1M] Một giải thuật có dùng stack được chọn để giải quyết bài toán chuyển biểu thức dạng trung tố sang hậu tố. Số lượng toán tử nhiều nhất tại một thời điểm trên stack khi chuyển biểu thức trung tố  $4 + 3 * (6 * 3 - 12)$  sang hậu tố là
- 1
  - 2
  - 3
  - 4
17. [2.2E] Xét một Danh sách liên kết đơn được biểu diễn bởi 2 con trỏ head, tail, lần lượt chứa địa chỉ của Node đầu tiên và Node cuối cùng trong Danh sách liên kết. Giả sử hiện tại Danh sách chỉ có 1 phần tử. Phát biểu nào sau đây là đúng sau khi xóa 1 Node ở vị trí cuối cùng?
- head = NULL
  - tail = NULL
  - head = NULL; tail = NULL; count = 0
  - head = NULL; tail = NULL;

18. [3.1M] Cho một min-heap được biểu diễn dưới dạng array như sau: 1, 4, 7, 8, 10, 12, 13  
Đâu là một biểu diễn ĐÚNG cho trạng thái của min-heap sau khi xóa một phần tử ra khỏi min-heap
- 4, 7, 8, 10, 13, 12
  - 4, 7, 8, 13, 10, 12
  - 4, 8, 13, 7, 10, 12
  - 4, 8, 7, 13, 10, 12
19. [3.1M] Với mỗi dãy khoá dưới đây, lần lượt thêm vào một bảng băm (rỗng) có kích thước là 19, với hàm băm là  $h(k) = k\%19$ , dãy khoá nào xảy ra ít đụng độ nhất?
- 0, 19, 57
  - 19, 20, 21, 22, 23, 46, 47, 57
  - 19, 20, 21, 22, 23, 40, 41, 57
  - 19, 20, 21, 39, 40
20. [1.1E] Mỗi biểu thức sau biểu diễn số thao tác của một giải thuật nhất định.  
 $P = n + 2\log(n) - 6$   
 $Q = (n^2 + 1)(n/2 + 1)$   
 $R = 4n * \log(n) + 1$   
 $S = n * (n^2 - 1)/2$   
Thứ tự nào sau đây biểu diễn đúng về độ phức tạp của các biểu thức trên
- $P < R < S = Q$
  - $P < S = Q < R$
  - $P < R < S < Q$
  - $P = S < R < Q$
21. [2.1M] Lần lượt thêm các số sau vào một cây AVL rỗng: 50, 23, 70, 19, 29, 65, 83, 25, 35, 53. Sau khi thêm xong, lần lượt xoá các số sau ra khỏi cây AVL: 19, 23, 83. Liệt kê các Node nằm ở tầng 2 (level 2) trong cây cuối cùng thu được theo thứ tự từ trái sang phải.
- 25, 29, 50, 70
  - 25, 35, 53, 70
  - 25, 50, 53, 70
  - 29, 35, 65, 70
22. [2.1M] Cho một danh sách liên kết đôi rỗng. Lần lượt thêm các số nguyên sau vào đầu danh sách liên kết đôi: 20, 22, 1, 3, 5, 7. Sau đó, lần lượt thêm các số nguyên sau vào cuối danh sách liên kết đôi: 20, 21, 22, 1, 3, 5, 7. Cho 2 con trỏ head, tail lần lượt chứa địa chỉ của Node đầu tiên và Node cuối cùng. Cho con trỏ temp chứa địa chỉ của Node đang chứa số 21. Cách làm nào sau đây cho ta đi đến vị trí của Node chứa số 1 (bất kì số 1 nào trong hai số 1) với số lần duyệt qua các Node khác ít nhất?
- Từ head duyệt về phía cuối danh sách
  - Từ tail duyệt về phía đầu danh sách
  - Từ temp duyệt về phía cuối danh sách
  - Từ temp duyệt về phía đầu danh sách
23. [2.1E] Phát biểu nào là đúng về điểm khác nhau của 1 Node trong Danh sách liên kết đôi so với danh sách liên kết đơn?
- Node có 2 trường dữ liệu.
  - Node có 2 liên kết.
  - Node có 2 con trỏ head1 và head2.
  - Node có 2 con trỏ tail1 và tail2.
24. [2.1M] Cây B (B-Tree) bậc  $m = 4$  và chiều cao bằng 4, có tối đa bao nhiêu entry
- 255
  - 160
  - 127
  - 64
25. [2.1E] Cho biết nhận định nào sau đây là SAI về cây AVL
- Cây AVL là cây tìm kiếm nhị phân (Binary Search Tree)
  - Cây AVL là cây cân bằng
  - Cây AVL là cây đầy đủ hoặc gần đầy đủ
- Nhận định i) sai
  - Nhận định ii) sai
  - Nhận định iii) sai
  - Không có nhận định nào sai
26. [2.1E] Để duyệt đồ thị theo chiều rộng (breadth-first search), người ta thường dùng cấu trúc dữ liệu nào sau đây?
- Stack (xếp chồng)
  - Queue (hàng đợi)**
  - Linked list (danh sách liên kết)
  - Graph (một đồ thị khác)
27. [2.1E] Sự khác biệt giữa Queue và Stack là
- Queue sử dụng 2 đầu của cấu trúc, Stack chỉ dùng 1
  - Stack cần phải cấp phát động, Queue thì không
  - Queue cần phải cấp phát động, Stack thì không
  - Stack sử dụng 2 đầu của cấu trúc, Queue chỉ dùng 1
28. [2.1E] Cho biết những nhận định nào sau đây là ĐÚNG về cây B (B-Tree) bậc  $m$
- Mỗi nút phải có ít nhất 2 cây con không rỗng
  - Mỗi nút có nhiều nhất  $m - 1$  dữ liệu (entry)
  - Tất cả các nút lá nằm trên cùng một mức
- Chỉ có i) và iii) đúng
  - Chỉ có i) và ii) đúng
  - Chỉ có ii) và iii) đúng
  - Cả ba nhận định trên đều đúng

29. [2.1E] Đây là độ phức tạp của thao tác ReHeapUp?

- (a)  $O(\log n)$
- (b)  $O(n)$
- (c)  $O(n \log n)$
- (d)  $O(1)$

30. [2.1M] Biết rằng duyệt tiền thứ tự và duyệt hậu thứ tự một cây nhị phân cho kết quả ngược nhau. Ví dụ, duyệt tiền thứ tự là (A,B,C,D), duyệt hậu thứ tự là (D,C,B,A), nhận định nào sau đây đúng nhất

- (a) Bất cứ nút nào cũng không có cây con bên trái
- (b) Bất cứ nút nào cũng không có cây con bên phải
- (c) Cây có chiều cao đúng bằng tổng số nút
- (d) Cây nhị phân hoặc là rỗng hoặc chỉ có một nút

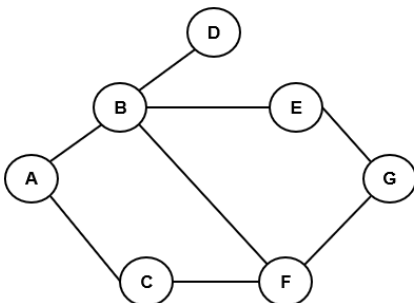
31. [2.1E] Đối với cây AVL, hệ số cân bằng (chênh lệch chiều cao cây con) ở mỗi nút trong cây có giá trị nằm trong đoạn

- (a)  $[0, 1]$
- (b)  $[-1, 1]$
- (c)  $[1, 2]$
- (d)  $[-2, 2]$

32. [3.1H] Cho cây nhị phân có các nút A, B, C, D, E, F, G. Duyệt tiền thứ tự (PreOrder) trên cây nhị phân thu được (A,B,C,D,E,F,G). Khi duyệt theo trung thứ tự (InOrder), kết quả có thể là

- (a) (C,A,B,D,E,F,G)
- (b) (A,B,C,D,E,F,G)
- (c) (D,A,C,E,F,B,G)
- (d) (A,D,C,F,E,G,B)

33. [2.2M] Cho đồ thị:



Đáp án nào sau đây có thể là số đỉnh phải duyệt khi dùng thuật toán DFS đi từ đỉnh A để tìm đỉnh F (Kể cả lần duyệt đỉnh F)

- (a) 2
- (b) 6
- (c) 5
- (d) Tất cả đều sai

34. [2.1E] Đây là độ phức tạp của giải thuật Heapsort ở trường hợp tốt nhất (best-case)

- (a)  $O(n \log n)$
- (b)  $O(\log n)$
- (c)  $O(n)$
- (d)  $O(n^2)$

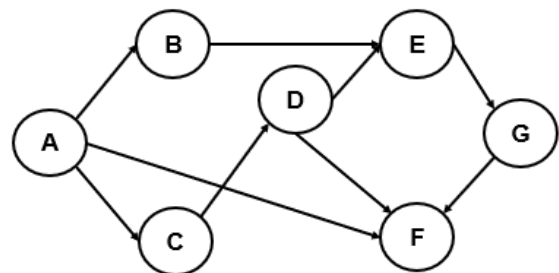
35. [2.1E] Câu nào sau đây là KHÔNG đúng về Stack?

- (a) Stack là cấu trúc dữ liệu mà các phần tử chỉ được thêm hoặc xóa ở một đầu duy nhất
- (b) Stack chỉ có thể hiện thực dưới dạng mảng
- (c) Stack tuân theo cấu trúc LIFO
- (d) Stack không hỗ trợ thao tác truy cập các phần tử nằm giữa cấu trúc

36. [3.1E] Phát biểu nào sau đây là đúng về tình trạng độn độ trong phương pháp băm?

- (a) Độn độ là tình trạng 2 khoá khác nhau cho ra 2 giá trị băm khác nhau.
- (b) Độn độ là tình trạng 2 khoá giống nhau cho ra 2 giá trị băm khác nhau.
- (c) Độn độ là tình trạng 2 khoá giống nhau cho ra 2 giá trị băm giống nhau.
- (d) Độn độ là tình trạng 2 khoá khác nhau cho ra 2 giá trị băm giống nhau.

37. [2M] Cặp công việc và thuật toán nào dưới đây có thể đưa đến trường hợp xấu nhất?



- (a) Đi từ đỉnh A đến đỉnh F bằng thuật toán BFS
- (b) Đi từ đỉnh B đến đỉnh F bằng thuật toán DFS
- (c) Đi từ đỉnh C đến đỉnh F bằng thuật toán DFS
- (d) Đi từ đỉnh C đến đỉnh F bằng thuật toán BFS

38. [2.2M] Cho dãy (22, 67, 13, 99, 4, 35, 44, 64, 17, 18). Chọn phần tử có index  $N/2$  làm pivot. Sau bước chia đầu tiên:

- (a) (18, 22, 13, 17, 4, 35, 44, 64, 99, 67)
- (b) (22, 18, 13, 17, 4, 35, 44, 64, 67, 99)
- (c) (22, 18, 13, 17, 4, 35, 44, 64, 99, 67)
- (d) (22, 18, 13, 17, 4, 35, 64, 44, 99, 67)

39. [2.1M] Một giải thuật dùng stack để giải quyết bài toán kiểm tra tính hợp lệ của một dãy đóng-mở ngoặc. Số lượng phần tử lớn nhất trên stack tại một thời điểm khi sử dụng giải thuật này với chuỗi đóng-mở ngoặc  $((()())())$  là?
- 1
  - 2
  - 3
  - Từ 4 trở lên
40. [2.2H] Sắp xếp dãy (77, 61, 24, 3, 68, 16, 49, 53, 64, 25) với Shell sort. Sau bước đầu tiên với step 3:
- (24, 61, 77, 3, 16, 68, 49, 53, 64, 25)
  - (3, 25, 49, 77, 53, 61, 68, 16, 24, 64)
  - (3, 53, 16, 25, 61, 24, 49, 68, 64, 77)
  - Tất cả đều sai

## II. Trả lời ngắn (30p)

41. [1E] Cho đoạn chương trình:

```
1 for(int i = 0; i < n; i++) i *= k;
```

Hãy cho biết độ phức tạp Big(O) của đoạn chương trình trên: .....

42. [2.2E] Cho cấu trúc dữ liệu của một node trên một cây nhị phân như sau:

```
1 template <typename E> class BinNode {
2 public:
3 virtual E& element() = 0; //return the node's value
4 virtual BinNode* left() const = 0; //return the node's left child
5 virtual BinNode* right() const = 0; //return the node's right child
6 };
7 template <typename E>
8 int height(BinNode<E> *pRoot) {
9 if (*(a)) /*(b)*/
10 return /*(c)*/;
11 }
```

Hãy điền vào các chỗ trống để hoàn thành hàm tính chiều cao của cây nhị phân từ nút gốc root

- .....
- .....
- .....

43. [2.2H] Giả sử kiểu E chấp nhận các phép so sánh > và <; hàm isBST trả về true khi root là nút gốc của một cây nhị phân tìm kiếm và false khi ngược lại. Hãy điền vào các chỗ trống của hàm checkbst để hàm isBST thực hiện đúng chức năng đã nêu?

```
1 template <typename E>
2 bool checkBST(BinNode<E> *pRoot, BinNode<E> *pLeft = nullptr, BinNode<E> *pRight = nullptr) {
3 if (!pRoot) return true;
4 if (pLeft && /*(a)*/) return false;
5 if (pRight && /*(b)*/) return false;
6 return /*(c)*/;
7 }
8 template <typename E>
9 bool isBST(BinNode<E> *pRoot) {
10 return checkBST(pRoot);
11 }
```

- .....
- .....
- .....

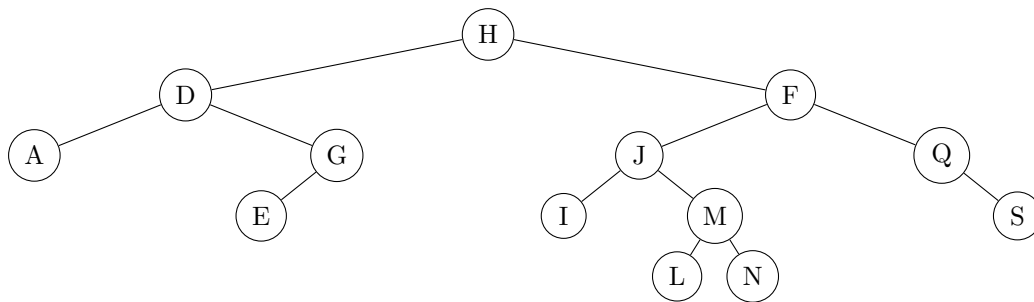
44. [3.1H] Lần lượt thêm các phần tử nguyên 3, 7, 9, 23, 45, 1, 5, 14, 25, 24, 13, 11, 8, 19, 4, 31, 35, 56 vào một cây B-Tree rỗng (bậc  $m = 5$ ). Hãy liệt kê (từ trái sang phải) các node có tổng giá trị các entry nằm trong khoảng [10, 30].

.....

45. [3.1H] Cho max-heap mang các giá trị số nguyên như sau [29, 20, 10, 15, 18, 9, 5, 13, 2, 4, 15]. Lần lượt lấy  $m$  phần tử từ đỉnh của max-heap nói trên để tổng của  $m$  phần tử này không vượt quá 70. Hãy cho biết giá trị của  $m$  và ghi lại heap sau khi đã lấy số phần tử nói trên.

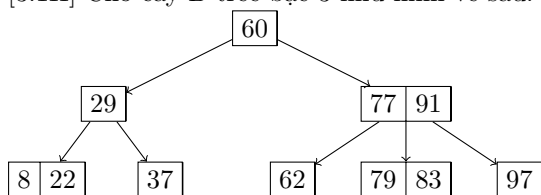
.....

46. [3.1M] Cho cây nhị phân như sau:



Vẽ lại cây AVL này sau khi xóa giá trị **I** khỏi cây:

47. [3.1H] Cho cây B-tree bậc 3 như hình vẽ sau.



Chèn vào cây 3 giá trị 72, 41, 39 (theo thứ tự trên). Vẽ trạng thái cây cuối cùng.

48. [3.1M] Lần lượt thêm các phần tử số nguyên mang giá trị 2, 1, 4, 5, 9, 3, 6, 7 vào một cây AVL rỗng. Hãy cho biết tiền thứ tự (pre-order NLR) được duyệt ra từ cây.

.....

49. [3.1M] Cho một cây AVL có tiền thứ tự (NLR) là 50 40 30 10 45 60 55. Sau khi xóa phần tử 50, cây AVL có tổng các giá trị trên các phần tử ở mức (level) 2 của cây là bao nhiêu?

.....

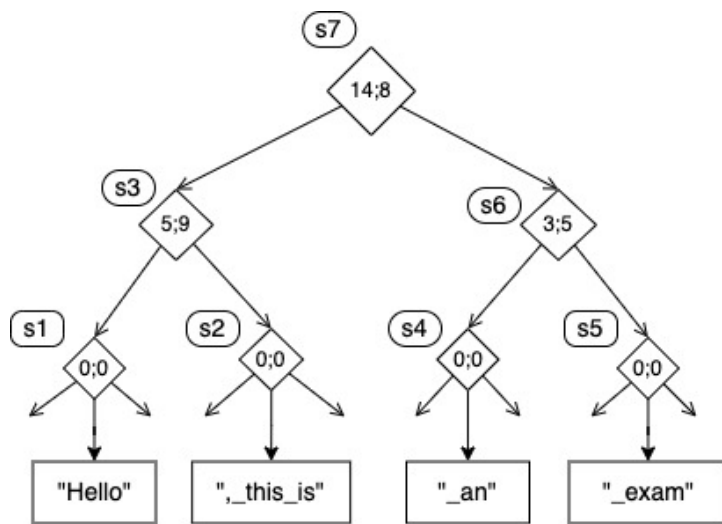
50. [3.2M] Cho dãy (22, 67, 13, 99, 4, 35, 44, 64, 17, 18). Chọn phần tử có index  $N/2$  làm pivot.

Dãy trên sau bước chia đầu tiên: .....

### III. Câu hỏi Harmony (10p)

Các câu hỏi sau được dùng để Harmony với điểm BTL 2 – Phần ConcatStringTree.

Sử dụng cấu trúc dữ liệu cây tìm kiếm nhị phân để biểu diễn một chuỗi ký tự như hình bên dưới:



Cho trước hiện thực của class Node như sau:

```

1 class BSTString; // forward declaration
2 class Node {
3 private:
4 string *data;
5 int lLen, rLen;
6 Node *left, *right;
7 friend class BSTString;
8 public:
9 Node(string * data=NULL, int lLen=0, int rLen=0, Node * left=NULL, Node * right=NULL)
10 : data(data), lLen(lLen), rLen(rLen), left(left), right(right) {}
11 };

```

Trong đó:

- data: là con trỏ đến chuỗi ký tự. Node trung gian (internal node) có data = NULL.
- left, right: lần lượt là con trỏ chứa địa chỉ Node gốc của cây con bên trái và cây con bên phải.
- lLen, rLen: lần lượt là độ dài của chuỗi được biểu diễn bởi cây con bên trái và cây con bên phải.

Sinh viên xem thêm hình trên về ví dụ của Node. Một Node trong hình được biểu diễn bằng một hình thoi với định dạng <lLen> và <rLen> ngăn cách nhau bởi 1 dấu chấm phẩy.

Gọi class biểu diễn chuỗi như hình là BSTString. Cho trước phần định nghĩa của class BSTString như sau:

```

1 class BSTString {
2 private:
3 Node* root;
4 public:
5 BSTString(Node * root=NULL): root(root) {}
6 char getRecr(Node * root, int index);
7 char get(int index);
8 BSTString concat(const BSTString & otherStr);
9 Node* reverseRecr(Node * root) const;
10 BSTString reverse() const;
11 };

```

51. [2.2E] (3p) Phương thức get trả về ký tự trong chuỗi tại vị trí “index”, với “index” luôn nằm trong khoảng hợp lệ. Cho trước phần hiện thực phương thức get như sau:

```

1 char BSTString::getRecr(Node * root, int index) {
2 if (root->left == NULL && root->right == NULL) {
3 // Handle for leaf nodes, please ignore this block
4 }
5 else if (index < root->lLen) {
6 return *(a)*/;
7 }
8 else {
9 return *(b)*/;
10 }
11 }
12 char BSTString::get(int index) {
13 return getRecr(this->root, index);
14 }

```

Ghi lệnh/biểu thức thích hợp cho các chỗ trống (a) và (b).

- (a) .....  
 (b) .....

52. [2.2E] (3p) Phương thức concat thực hiện việc nối 2 chuỗi BSTString và trả về một đối tượng BSTString mới. Ví dụ: Trong hình trên, khi nối chuỗi s1 với s2, có một node mới được tạo ra là (5;9) và kết nối với s1, s2. Một đối tượng BSTString được tạo mới là s3 có node gốc là (5;9). Cho trước phần hiển thực phương thức concat như sau:

```

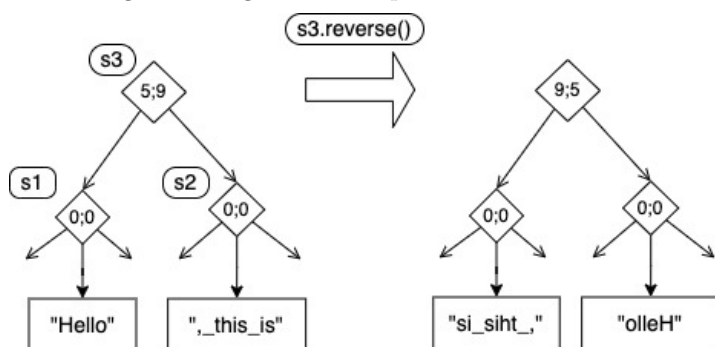
1 BSTString BSTString::concat(const BSTString & otherStr) {
2 Node * newRoot = new Node(
3 /*(a)*/,
4 /* Please ignore this parameter */,
5 /* Please ignore this parameter */,
6 /*(b)*/,
7 /*(c)*/
8);
9 return BSTString(newRoot);
10 }

```

Điền các thông số thích hợp vào (a), (b), (c)

- (a).....  
 (b).....  
 (c).....

53. [2.2M] (4p) Phương thức reverse trả về đối tượng BSTString mới biểu diễn một chuỗi nghịch đảo của chuỗi gốc. Các node trong đối tượng trả về đều phải được tạo mới.



Cho trước phần hiển thực phương thức reverse như sau:

```

1 Node* BSTString::reverseRecr(Node * root) const {
2 if (root == NULL) return NULL;
3 else if (root->left == NULL && root->right == NULL) { /* Handle for leaf nodes, please ignore this
4 block */ }
5 else { /* Handle internal nodes
6 Node * newRoot = new Node();
7 newRoot->left = /* (a) */;
8 newRoot->right = /* (b) */;
9 newRoot->lLen = /* (c) */;
10 newRoot->rLen = /* (d) */;
11 return newRoot;
12 }
13 }
14 BSTString BSTString::reverse() const{
15 Node * newRoot = reverseRecr(this->root);
16 return BSTString(newRoot);
17 }

```

Điền các thông số thích hợp vào (a), (b), (c), (d)

- (a).....  
 (b).....  
 (c).....  
 (d).....



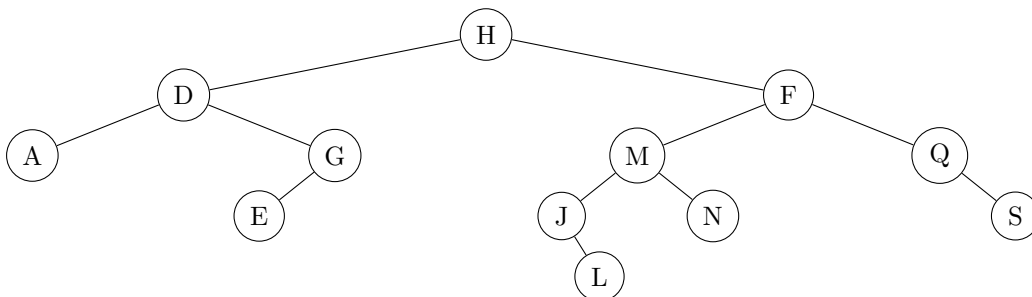
# Solution 2211

## I. Trắc nghiệm (60p)

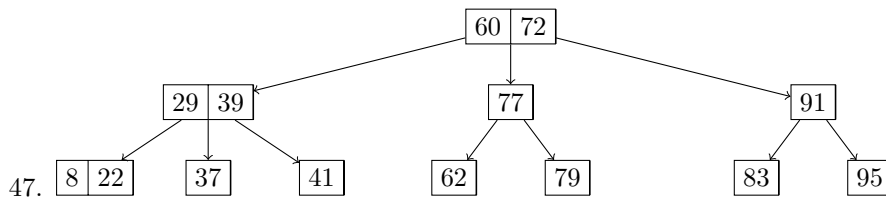
- |         |         |
|---------|---------|
| 1. (a)  | 21. (b) |
| 2. (c)  | 22. (d) |
| 3. (c)  | 23. (b) |
| 4. (b)  | 24. (a) |
| 5. (d)  | 25. (c) |
| 6. (d)  | 26. (b) |
| 7. (d)  | 27. (a) |
| 8. (c)  | 28. (c) |
| 9. (a)  | 29. (a) |
| 10. (a) | 30. (c) |
| 11. (c) | 31. (b) |
| 12. (c) | 32. (b) |
| 13. (c) | 33. (b) |
| 14. (a) | 34. (a) |
| 15. (b) | 35. (b) |
| 16. (d) | 36. (d) |
| 17. (d) | 37. (c) |
| 18. (d) | 38. (c) |
| 19. (b) | 39. (c) |
| 20. (a) | 40. (c) |

## II. Trả lời ngắn (30p)

41.  $O(\log_k(n))$
42. (a) `pRoot == NULL`  
(b) `return 0`  
(c) dòng 15: `1 + max(height(root->left()), height(root->right()))`
43. (a) `root->element() < left->element()` (b) `root->element() > right->element()` (c) `checkbst(root->left(), left, root)`  
&& `checkbst(root->right(), root, right)`
44. `[14], [7, 8], [5, 9], [11, 13]`.
45. `[15, 15, 10, 13, 2, 9, 5, 4]`



46.



48. NLR: 4 2 1 3 6 5 9 7

49.  $10 + 45 + 60 = 115$

50. (22, 18, 13, 17, 4, 35, 44, 64, 99, 67)

### III. Câu hỏi Harmony (10p)

51. `/*(a)*/ this->getRecr(root->left, index)`

`/*(b)*/ this->getRecr(root->right, (index - root->lLen))`

52. `/*(a)*/ NULL`

`/*(b)*/ this->root`

`/*(c)*/ otherStr.root`

53. `/*(a)*/ reverseRecr(root->right)`

`/*(b)*/ reverseRecr(root->left)`

`/*(c)*/ (newRoot->left->data != NULL) ? (newRoot->left->data->length()) : (newRoot->left->lLen + newRoot->left->rLen)`

`/*(d)*/ (newRoot->right->data != NULL) ? (newRoot->right->data->length()) : (newRoot->right->lLen + newRoot->right->rLen)`

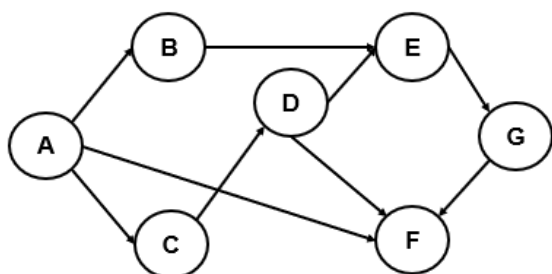
|                                                |              |                                               |                 |
|------------------------------------------------|--------------|-----------------------------------------------|-----------------|
| <b>Giảng viên ra đề:</b><br>(Chữ ký và Họ tên) | (Ngày ra đề) | <b>Người phê duyệt:</b><br>(Chữ ký và họ tên) | (Ngày duyệt đề) |
|------------------------------------------------|--------------|-----------------------------------------------|-----------------|

|                                                                                                                                                                                                                                         |                    |                                |                  |            |           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|--------------------------------|------------------|------------|-----------|
| <br><b>TRƯỜNG ĐH BÁCH KHOA - ĐHQG-HCM</b><br><b>KHOA KH &amp; KT MÁY TÍNH</b>                                                                          | <b>THI CUỐI KỲ</b> |                                | Học kỳ / Năm học | 1          | 2022-2023 |
|                                                                                                                                                                                                                                         |                    |                                | Ngày thi         | 21-12-2022 |           |
|                                                                                                                                                                                                                                         | Môn học            | Cấu trúc dữ liệu và giải thuật |                  |            |           |
|                                                                                                                                                                                                                                         | Mã môn học         | CO2003                         |                  |            |           |
|                                                                                                                                                                                                                                         | Thời lượng         | 80 phút                        | Mã đề            | 2212       |           |
| Ghi chú: - Sinh viên không sử dụng tài liệu.<br>- Sinh viên làm trực tiếp trên đề và nộp lại.<br>- Quy ước chọn ưu tiên trong phép duyệt là thứ tự alphabet.<br>- Quy ước so sánh trên cây nhị phân dùng ký tự là theo thứ tự alphabet. |                    |                                |                  |            |           |

## I. Trắc nghiệm (60p)

- [2.1E] Đối với cây AVL, hệ số cân bằng (chênh lệch chiều cao cây con) ở mỗi nút trong cây có giá trị nằm trong đoạn
  - [0, 1]
  - [-1, 1]
  - [1, 2]
  - [-2, 2]
- [3.1M] Duyệt trung thứ tự cây nhị phân cho kết quả là (A,B,C,D,E,F,G), duyệt hậu thứ tự cho kết quả là (B,D,C,A,F,G,E), hãy cho biết cây con bên phải của nút gốc có tổng cộng bao nhiêu nút
  - 2
  - 3
  - 4
  - 5

- [2M] Cặp công việc và thuật toán nào dưới đây có thể đưa đến trường hợp xấu nhất?



- Đi từ đỉnh A đến đỉnh F bằng thuật toán BFS
- Đi từ đỉnh B đến đỉnh F bằng thuật toán DFS
- Đi từ đỉnh C đến đỉnh F bằng thuật toán DFS
- Đi từ đỉnh C đến đỉnh F bằng thuật toán BFS

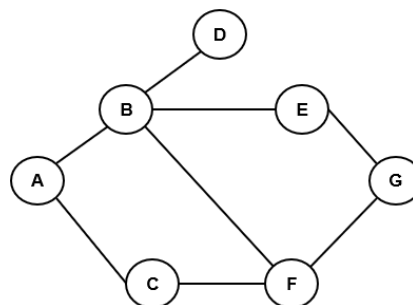
- [2.1M] Cho biết nhận định nào dưới đây về cây B (B-Tree) bậc m là KHÔNG đúng

- Nút gốc có nhiều nhất m cây con
- Tất cả các nút lá nằm trên cùng một mức
- Các khóa trong nút gốc được sắp xếp theo thứ tự
- Nút trong có ít nhất  $m/2 + 1$  cây con khác rỗng (nếu m chẵn) hoặc  $m/2 - 1$  cây con khác rỗng (nếu m lẻ)

- [2.1E] Cho biết nhận định nào sau đây là SAI về cây AVL
  - Cây AVL là cây tìm kiếm nhị phân (Binary Search Tree)
  - Cây AVL là cây cân bằng
  - Cây AVL là cây đầy đủ hoặc gần đầy đủ

- Nhận định i) sai
- Nhận định ii) sai
- Nhận định iii) sai
- Không có nhận định nào sai

- [2.2M] Cho đồ thị:



Đáp án nào sau đây có thể là số đỉnh phải duyệt khi dùng thuật toán DFS đi từ đỉnh A để tìm đỉnh F (Kể cả lần duyệt đỉnh F)

- 2
- 6
- 5
- Tất cả đều sai

7. [3.1M] Với mỗi dãy khoá dưới đây, lần lượt thêm vào một bảng băm (rỗng) có kích thước là 19, với hàm băm là  $h(k) = k \% 19$ , dãy khoá nào xảy ra ít đụng độ nhất?

(a) 0, 19, 57  
(b) 19, 20, 21, 22, 23, 46, 47, 57  
(c) 19, 20, 21, 22, 23, 40, 41, 57  
(d) 19, 20, 21, 39, 40

8. [2.1M] Lần lượt thêm các số sau vào một cây AVL rỗng: 50, 23, 70, 19, 29, 65, 83, 25, 35, 53. Sau khi thêm xong, lần lượt xoá các số sau ra khỏi cây AVL: 19, 23, 83. Liệt kê các Node nằm ở tầng 2 (level 2) trong cây cuối cùng thu được theo thứ tự từ trái sang phải.

(a) 25, 29, 50, 70  
(b) 25, 35, 53, 70  
(c) 25, 50, 53, 70  
(d) 29, 35, 65, 70

9. [1.2M] Cho đoạn chương trình sau:

```
1 int func(int x){
2 if (x == 0) return 1;
3 if (x == 1) return func(0);
4 return func(x - 1) + x;
5 }
```

Số lần phép so sánh được thực thi khi gọi hàm  $f(N)$  (với  $N > 0$ ) là:

(a)  $N$   
(b)  $2N$   
(c)  $2N + 1$   
(d)  $N^2$

10. [3.1H] Xây dựng cây B (B-Tree) bậc 3 từ dãy khoá sau (18, 4, 2, 46, 48, 29, 30). Hãy cho biết chèn các khoá nào dưới đây vào cây B ở trên, thì sẽ xảy ra việc tách nút (split node)

(a) 1, 32, 53  
(b) 1, 19, 20  
(c) 1, 19, 32  
(d) 1, 19, 32, 53

11. [3.1H] Biết rằng duyệt tiền thứ tự một cây nhị phân cho kết quả là (A,B,C,D,E,K,F,G), kết quả duyệt trung thứ tự là (B,C,A,D,K,E,F,G). Hãy cho biết kết quả duyệt hậu thứ tự

(a) C,B,K,G,F,E,D,A  
(b) C,B,K,G,E,F,D,A  
(c) C,B,G,K,F,E,D,A  
(d) Cả ba đáp án còn lại đều sai

12. [2.2E] Xét một Danh sách liên kết đơn được biểu diễn bởi 2 con trỏ head, tail, lần lượt chứa địa chỉ của Node đầu tiên và Node cuối cùng trong Danh sách liên kết. Giả sử hiện tại Danh sách chỉ có 1 phần tử. Phát biểu nào sau đây là đúng sau khi xoá 1 Node ở vị trí cuối cùng?

(a) head = NULL  
(b) tail = NULL  
(c) head = NULL; tail = NULL; count = 0  
(d) head = NULL; tail = NULL;

13. [1.1E] Mỗi biểu thức sau biểu diễn số thao tác của một giải thuật nhất định.

$P = n + 2\log(n) - 6$   
 $Q = (n^2 + 1)(n/2 + 1)$   
 $R = 4n * \log(n) + 1$   
 $S = n * (n^2 - 1)/2$

Thứ tự nào sau đây biểu diễn đúng về độ phức tạp của các biểu thức trên

(a)  $P < R < S = Q$   
(b)  $P < S = Q < R$   
(c)  $P < R < S < Q$   
(d)  $P = S < R < Q$

14. [2.1E] Để duyệt đồ thị theo chiều rộng (breadth-first search), người ta thường dùng cấu trúc dữ liệu nào sau đây?

(a) Stack (xếp chồng)  
(b) Queue (hàng đợi)  
(c) Linked list (danh sách liên kết)  
(d) Graph (một đồ thị khác)

15. [2.1E] Phát biểu nào sau đây là đúng khi nói về một Node trong danh sách liên kết?

(a) Node có thể có nhiều trường dữ liệu.  
(b) Node phải có 1 trường dữ liệu thuộc kiểu số nguyên.  
(c) Node phải có 1 trường dữ liệu thuộc kiểu T.  
(d) Node có thể không có liên kết đến Node khác.

16. [2.1M] Cho một danh sách liên kết đôi rỗng. Lần lượt thêm các số nguyên sau vào đầu danh sách liên kết đôi: 20, 22, 1, 3, 5, 7. Sau đó, lần lượt thêm các số nguyên sau vào cuối danh sách liên kết đôi: 20, 21, 22, 1, 3, 5, 7. Cho 2 con trỏ head, tail lần lượt chứa địa chỉ của Node đầu tiên và Node cuối cùng. Cho con trỏ temp chứa địa chỉ của Node đang chứa số 21. Cách làm nào sau đây cho ta đi đến vị trí của Node chứa số 1 (bất kì số 1 nào trong hai số 1) với số lần duyệt qua các Node khác ít nhất?

(a) Từ head duyệt về phía cuối danh sách  
(b) Từ tail duyệt về phía đầu danh sách  
(c) Từ temp duyệt về phía cuối danh sách  
(d) Từ temp duyệt về phía đầu danh sách

17. [2M] Cho cây AVL với 2 nút, trong đó nút gốc có giá trị 15, nút còn lại có giá trị 20. Khi chèn thêm nút có giá trị 19, cần phải
- Thực hiện phép quay đơn (single rotation) về bên trái
  - Thực hiện phép quay đơn về bên phải
  - Thực hiện phép quay kép (double rotation)
  - Không cần thực hiện phép quay
18. [2.1M] Một giải thuật có dùng stack được chọn để giải quyết bài toán chuyển biểu thức dạng trung tố sang hậu tố. Số lượng toán tử nhiều nhất tại một thời điểm trên stack khi chuyển biểu thức trung tố  $4 + 3 * (6 * 3 - 12)$  sang hậu tố là
- 1
  - 2
  - 3
  - 4
19. [2.1E] Phát biểu nào là đúng về điểm khác nhau của 1 Node trong Danh sách liên kết đôi so với danh sách liên kết đơn?
- Node có 2 trường dữ liệu.
  - Node có 2 liên kết.
  - Node có 2 con trỏ head1 và head2.
  - Node có 2 con trỏ tail1 và tail2.
20. [2.1M] Một giải thuật dùng stack để giải quyết bài toán kiểm tra tính hợp lệ của một dãy đóng-mở ngoặc. Số lượng phần tử lớn nhất trên stack tại một thời điểm khi sử dụng giải thuật này với chuỗi đóng-mở ngoặc  $((()())())$  là?
- 1
  - 2
  - 3
  - Từ 4 trở lên
21. [2.2H] Sắp xếp dãy (77, 61, 24, 3, 68, 16, 49, 53, 64, 25) với Shell sort. Sau bước đầu tiên với step 3:
- (24, 61, 77, 3, 16, 68, 49, 53, 64, 25)
  - (3, 25, 49, 77, 53, 61, 68, 16, 24, 64)
  - (3, 53, 16, 25, 61, 24, 49, 68, 64, 77)
  - Tất cả đều sai
22. [3.1M] Cho một min-heap được biểu diễn dưới dạng array như sau: 1, 4, 7, 8, 10, 12, 13  
Đâu là một biểu diễn ĐÚNG cho trạng thái của min-heap sau khi xóa một phần tử ra khỏi min-heap
- 4, 7, 8, 10, 13, 12
  - 4, 7, 8, 13, 10, 12
  - 4, 8, 13, 7, 10, 12
  - 4, 8, 7, 13, 10, 12
23. [2.1M] Biết rằng duyệt tiền thứ tự và duyệt hậu thứ tự một cây nhị phân cho kết quả ngược nhau. Ví dụ, duyệt tiền thứ tự là (A,B,C,D), duyệt hậu thứ tự là (D,C,B,A), nhận định nào sau đây đúng nhất
- Bất cứ nút nào cũng không có cây con bên trái
  - Bất cứ nút nào cũng không có cây con bên phải
  - Cây có chiều cao đúng bằng tổng số nút
  - Cây nhị phân hoặc là rỗng hoặc chỉ có một nút
24. [3.1M] Lần lượt thêm các khoá 5; 7; 12; 25; 36; 58 vào một bảng băm có kích thước là 11, sử dụng hàm băm  $h(k) = k \bmod 11$ . Nếu đụng độ xảy ra thì phương pháp giải quyết đụng độ được chọn là mở rộng địa chỉ (open addressing) với hàm dò tìm là  $p(k, i) = h(k) + 2 * i + 1$ . Vị trí của khoá 58 trong bảng băm là:
- 0
  - 3
  - 6
  - 9
25. [3.1H] Cho cây AVL có kết quả duyệt tiền thứ tự là (13, 10, 5, 4, 6, 11, 15, 16), kết quả duyệt trung thứ tự là (4, 5, 6, 10, 11, 13, 15, 16). Chèn thêm nút có giá trị 7 vào cây AVL ở trên, kết quả duyệt tiền thứ tự sau khi chèn giá trị 7 là
- 13, 5, 4, 6, 10, 7, 11, 15, 16
  - 13, 6, 5, 4, 10, 7, 11, 15, 16
  - 6, 13, 5, 4, 10, 7, 11, 15, 16
  - 6, 5, 4, 10, 7, 11, 13, 15, 16
26. [3.1E] Phát biểu nào sau đây là đúng về Quadratic probing?
- Là một phương pháp hiện thực hàm băm.
  - Có thể dẫn đến Primary clustering.
  - Có thể dẫn đến Secondary clustering.
  - Có thể dẫn đến Linear clustering và Secondary clustering.
27. [3.1H] Cho cây nhị phân có các nút A, B, C, D, E, F, G. Duyệt tiền thứ tự (PreOrder) trên cây nhị phân thu được (A,B,C,D,E,F,G). Khi duyệt theo trung thứ tự (InOrder), kết quả có thể là
- (C,A,B,D,E,F,G)
  - (A,B,C,D,E,F,G)
  - (D,A,C,E,F,B,G)
  - (A,D,C,F,E,G,B)
28. [2.1E] Biết rằng mảng A dùng để biểu diễn một max-heap và một chỉ số i. Biết rằng việc truy cập các phần tử trong mảng đều hợp lệ, biểu thức nào sau đây LUÔN ĐÚNG?
- $A[i] > A[2i + 1]$
  - $A[i] < A[2i + 2]$
  - $A[i] > A[2i]$
  - $A[i] < A[2i - 1]$

29. [2.1M] Cây B (B-Tree) bậc  $m = 4$  và chiều cao bằng 4, có tối đa bao nhiêu entry

- (a) 255
- (b) 160
- (c) 127
- (d) 64

30. [2E] Thông thường, để lưu trữ một ma trận số, người ta sử dụng một mảng 2 chiều kích thước  $m \times n$  để chứa các phần tử này. Giả sử, ta sử dụng một mảng một chiều kích thước  $N$  để chứa các phần tử của ma trận trên. Khẳng định nào sau đây là đúng khi so sánh 2 chương trình in mảng một chiều kích thước  $N$  và mảng 2 chiều kích thước  $m \times n$ ?

- (a) Chương trình in mảng hai chiều sẽ nhanh hơn hẳn chương trình còn lại
- (b) Chương trình in mảng một chiều sẽ nhanh hơn hẳn do có độ phức tạp  $O(N)$
- (c) Hai chương trình có tốc độ gần như tương đương nhau
- (d) Các đáp án trên đều sai

31. [2.1E] Sự khác biệt giữa Queue và Stack là

- (a) Queue sử dụng 2 đầu của cấu trúc, Stack chỉ dùng 1
- (b) Stack cần phải cấp phát động, Queue thì không
- (c) Queue cần phải cấp phát động, Stack thì không
- (d) Stack sử dụng 2 đầu của cấu trúc, Queue chỉ dùng 1

32. [2.1E] Cho biết những nhận định nào sau đây là ĐÚNG về cây B (B-Tree) bậc  $m$

- i) Mỗi nút phải có ít nhất 2 cây con không rỗng
- ii) Mỗi nút có nhiều nhất  $m - 1$  dữ liệu (entry)
- iii) Tất cả các nút lá nằm trên cùng một mức

- (a) Chỉ có i) và iii) đúng
- (b) Chỉ có i) và ii) đúng
- (c) Chỉ có ii) và iii) đúng
- (d) Cả ba nhận định trên đều đúng

33. [2.1E] Đây là độ phức tạp của giải thuật Heapsort ở trường hợp tốt nhất (best-case)

- (a)  $O(n \log n)$
- (b)  $O(\log n)$
- (c)  $O(n)$
- (d)  $O(n^2)$

34. [1M] Giải thuật sau tìm kiếm target trong mảng tăng dần **a**:

```
1 int i, low = 0, high = n - 1;
2 while (low < high) {
3 i = (low + high) / 2;
4 if (a[i] == target) return i;
5 else if (a[i] < target) low = i + 1;
6 else high = i - 1;
7 }
8 return -1;
```

Với  $n = 13$  và  $\text{target} > a[n-1]$ , hãy cho biết số lần thực hiện phép so sánh trong các câu lệnh if

- (a) 4
- (b) 6
- (c) 7
- (d) 8

35. [2.2M] Cho dãy (22, 67, 13, 99, 4, 35, 44, 64, 17, 18). Chọn phần tử có index  $N/2$  làm pivot. Sau bước chia đầu tiên:

- (a) (18, 22, 13, 17, 4, 35, 44, 64, 99, 67)
- (b) (22, 18, 13, 17, 4, 35, 44, 64, 67, 99)
- (c) (22, 18, 13, 17, 4, 35, 44, 64, 99, 67)
- (d) (22, 18, 13, 17, 4, 35, 64, 44, 99, 67)

36. [2.1E] Đây là độ phức tạp của thao tác ReHeapUp?

- (a)  $O(\log n)$
- (b)  $O(n)$
- (c)  $O(n \log n)$
- (d)  $O(1)$

37. [2.1M] Lần lượt thêm các số sau vào 1 cây AVL rỗng: 3, 10, 20, 9, 1, 5. Liệt kê các Node nằm ở tầng 2 (level 2) trong cây cuối cùng thu được theo thứ tự tăng dần.

- (a) 3, 5, 20
- (b) 3, 10
- (c) 1, 5, 20
- (d) 3, 5, 10

38. [3.1E] Phát biểu nào sau đây là đúng về tình trạng đụng độ trong phương pháp băm?

- (a) Đụng độ là tình trạng 2 khoá khác nhau cho ra 2 giá trị băm khác nhau.
- (b) Đụng độ là tình trạng 2 khoá giống nhau cho ra 2 giá trị băm khác nhau.
- (c) Đụng độ là tình trạng 2 khoá giống nhau cho ra 2 giá trị băm giống nhau.
- (d) Đụng độ là tình trạng 2 khoá khác nhau cho ra 2 giá trị băm giống nhau.

39. [2.1E] Câu nào sau đây là KHÔNG đúng về Stack?

- (a) Stack là cấu trúc dữ liệu mà các phần tử chỉ được thêm hoặc xóa ở một đầu duy nhất
- (b) Stack chỉ có thể hiện thực dưới dạng mảng
- (c) Stack tuân theo cấu trúc LIFO
- (d) Stack không hỗ trợ thao tác truy cập các phần tử nằm giữa cấu trúc

40. [2.1E] Mảng nào sau đây biểu diễn một min-heap?

- (a) 1, 2, 8, 4, 13, 6, 9, 15
- (b) 1, 2, 4, 13, 6, 8, 15, 9
- (c) 1, 2, 13, 15, 6, 4, 9, 8
- (d) 1, 2, 6, 4, 13, 8, 9, 15

## II. Trả lời ngắn (30p)

41. [1E] Cho đoạn chương trình:

```
1 for(int i = 0; i < n; i++) i *= k;
```

Hãy cho biết độ phức tạp Big(O) của đoạn chương trình trên: .....

42. [2.2E] Cho cấu trúc dữ liệu của một node trên một cây nhị phân như sau:

```
1 template <typename E> class BinNode {
2 public:
3 virtual E& element() = 0; //return the node's value
4 virtual BinNode* left() const = 0; //return the node's left child
5 virtual BinNode* right() const = 0; //return the node's right child
6 };
7 template <typename E>
8 int height(BinNode<E> *pRoot) {
9 if (/*(a)*/) /*(b)*/
10 return /*(c)*/;
11 }
```

Hãy điền vào các chỗ trống để hoàn thành hàm tính chiều cao của cây nhị phân từ nút gốc root

- (a) .....
- (b) .....
- (c) .....

43. [2.2H] Giả sử kiểu E chấp nhận các phép so sánh > và <; hàm isBST trả về true khi root là nút gốc của một cây nhị phân tìm kiếm và false khi ngược lại. Hãy điền vào các chỗ trống của hàm checkbst để hàm isBST thực hiện đúng chức năng đã nêu?

```
1 template <typename E>
2 bool checkBST(BinNode<E> *pRoot, BinNode<E> *pLeft = nullptr, BinNode<E> *pRight = nullptr) {
3 if (!pRoot) return true;
4 if (pLeft && /*(a)*/) return false;
5 if (pRight && /*(b)*/) return false;
6 return /*(c)*/;
7 }
8 template <typename E>
9 bool isBST(BinNode<E> *pRoot) {
10 return checkBST(pRoot);
11 }
```

- (a) .....
- (b) .....
- (c) .....

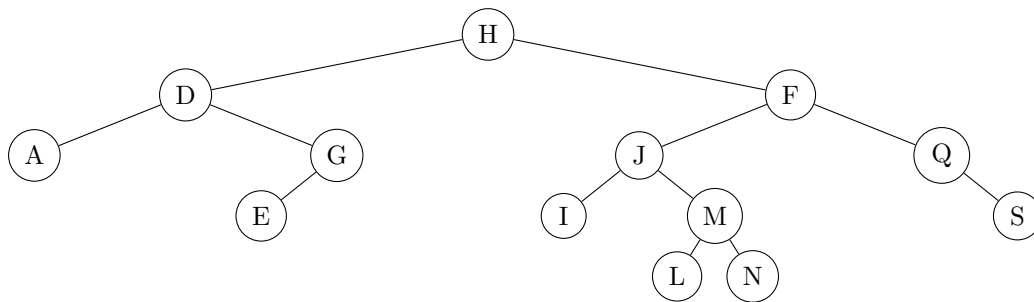
44. [3.1H] Lần lượt thêm các phần tử nguyên 3, 7, 9, 23, 45, 1, 5, 14, 25, 24, 13, 11, 8, 19, 4, 31, 35, 56 vào một cây B-Tree rỗng (bậc  $m = 5$ ). Hãy liệt kê (từ trái sang phải) các node có tổng giá trị các entry nằm trong khoảng [10, 30].

.....

45. [3.1H] Cho max-heap mang các giá trị số nguyên như sau [29, 20, 10, 15, 18, 9, 5, 13, 2, 4, 15]. Lần lượt lấy  $m$  phần tử từ đỉnh của max-heap nói trên để tổng của  $m$  phần tử này không vượt quá 70. Hãy cho biết giá trị của  $m$  và ghi lại heap sau khi đã lấy số phần tử nói trên.

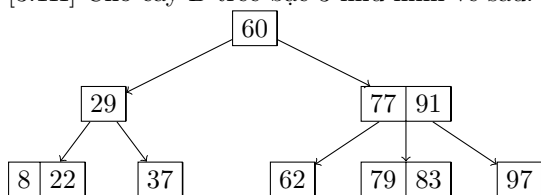
.....

46. [3.1M] Cho cây nhị phân như sau:



Vẽ lại cây AVL này sau khi xóa giá trị **I** khỏi cây:

47. [3.1H] Cho cây B-tree bậc 3 như hình vẽ sau.



Chèn vào cây 3 giá trị 72, 41, 39 (theo thứ tự trên). Vẽ trạng thái cây cuối cùng.

48. [3.1M] Lần lượt thêm các phần tử số nguyên mang giá trị 2, 1, 4, 5, 9, 3, 6, 7 vào một cây AVL rỗng. Hãy cho biết tiền thứ tự (pre-order NLR) được duyệt ra từ cây.

.....

49. [3.1M] Cho một cây AVL có tiền thứ tự (NLR) là 50 40 30 10 45 60 55. Sau khi xóa phần tử 50, cây AVL có tổng các giá trị trên các phần tử ở mức (level) 2 của cây là bao nhiêu?

.....

50. [3.2M] Cho dãy (22, 67, 13, 99, 4, 35, 44, 64, 17, 18). Chọn phần tử có index  $N/2$  làm pivot.

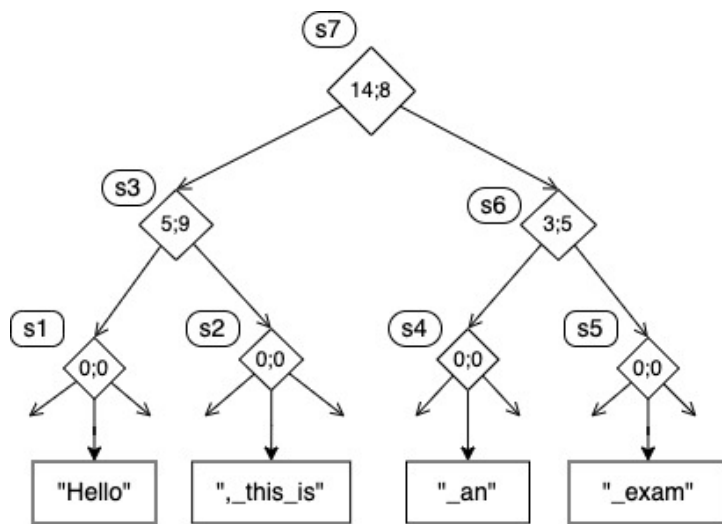
Dãy trên sau bước chia đầu tiên: .....

### III. Câu hỏi Harmony (10p)

Các câu hỏi sau được dùng để Harmony với điểm BTL 2 – Phần ConcatStringTree.

Sử dụng cấu trúc dữ liệu cây tìm kiếm nhị phân để biểu diễn một chuỗi ký tự như hình bên dưới:





Cho trước hiện thực của class Node như sau:

```

1 class BSTString; // forward declaration
2 class Node {
3 private:
4 string *data;
5 int lLen, rLen;
6 Node *left, *right;
7 friend class BSTString;
8 public:
9 Node(string * data=NULL, int lLen=0, int rLen=0, Node * left=NULL, Node * right=NULL)
10 : data(data), lLen(lLen), rLen(rLen), left(left), right(right) {}
11 };

```

Trong đó:

- data: là con trỏ đến chuỗi ký tự. Node trung gian (internal node) có data = NULL.
- left, right: lần lượt là con trỏ chứa địa chỉ Node gốc của cây con bên trái và cây con bên phải.
- lLen, rLen: lần lượt là độ dài của chuỗi được biểu diễn bởi cây con bên trái và cây con bên phải.

Sinh viên xem thêm hình trên về ví dụ của Node. Một Node trong hình được biểu diễn bằng một hình thoi với định dạng <lLen> và <rLen> ngăn cách nhau bởi 1 dấu chấm phẩy.

Gọi class biểu diễn chuỗi như hình là BSTString. Cho trước phần định nghĩa của class BSTString như sau:

```

1 class BSTString {
2 private:
3 Node* root;
4 public:
5 BSTString(Node * root=NULL): root(root) {}
6 char getRecr(Node * root, int index);
7 char get(int index);
8 BSTString concat(const BSTString & otherStr);
9 Node* reverseRecr(Node * root) const;
10 BSTString reverse() const;
11 };

```

51. [2.2E] (3p) Phương thức get trả về ký tự trong chuỗi tại vị trí “index”, với “index” luôn nằm trong khoảng hợp lệ. Cho trước phần hiện thực phương thức get như sau:

```

1 char BSTString::getRecr(Node * root, int index) {
2 if (root->left == NULL && root->right == NULL) {
3 // Handle for leaf nodes, please ignore this block
4 }
5 else if (index < root->lLen) {
6 return *(a)*/;
7 }
8 else {
9 return *(b)*/;
10 }
11 }
12 char BSTString::get(int index) {
13 return getRecr(this->root, index);
14 }

```

Ghi lệnh/biểu thức thích hợp cho các chỗ trống (a) và (b).

- (a) .....  
 (b) .....

52. [2.2E] (3p) Phương thức concat thực hiện việc nối 2 chuỗi BSTString và trả về một đối tượng BSTString mới. Ví dụ: Trong hình trên, khi nối chuỗi s1 với s2, có một node mới được tạo ra là (5;9) và kết nối với s1, s2. Một đối tượng BSTString được tạo mới là s3 có node gốc là (5;9). Cho trước phần hiển thực phương thức concat như sau:

```

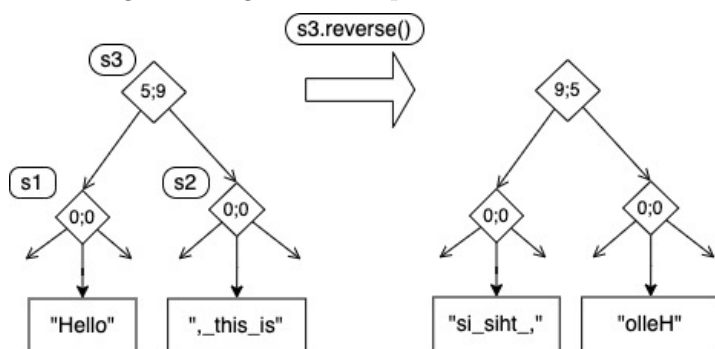
1 BSTString BSTString::concat(const BSTString & otherStr) {
2 Node * newRoot = new Node(
3 /*(a)*/,
4 /* Please ignore this parameter */,
5 /* Please ignore this parameter */,
6 /*(b)*/,
7 /*(c)*/
8);
9 return BSTString(newRoot);
10 }

```

Điền các thông số thích hợp vào (a), (b), (c)

- (a).....  
 (b).....  
 (c).....

53. [2.2M] (4p) Phương thức reverse trả về đối tượng BSTString mới biểu diễn một chuỗi nghịch đảo của chuỗi gốc. Các node trong đối tượng trả về đều phải được tạo mới.



Cho trước phần hiển thực phương thức reverse như sau:

```

1 Node* BSTString::reverseRecr(Node * root) const {
2 if (root == NULL) return NULL;
3 else if (root->left == NULL && root->right == NULL) { /* Handle for leaf nodes, please ignore this
4 block */ }
5 else { // Handle internal nodes
6 Node * newRoot = new Node();
7 newRoot->left = /* (a) */;
8 newRoot->right = /* (b) */;
9 newRoot->lLen = /* (c) */;
10 newRoot->rLen = /* (d) */;
11 return newRoot;
12 }
13 }
14 BSTString BSTString::reverse() const{
15 Node * newRoot = reverseRecr(this->root);
16 return BSTString(newRoot);
17 }

```

Điền các thông số thích hợp vào (a), (b), (c), (d)

- (a).....  
 (b).....  
 (c).....  
 (d).....

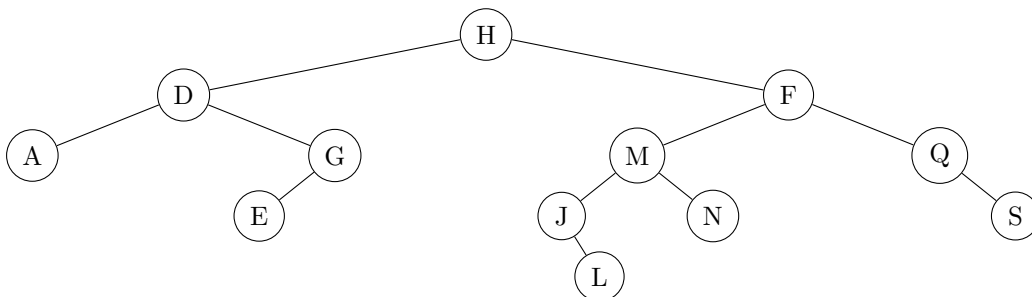
# Solution 2212

## I. Trắc nghiệm (60p)

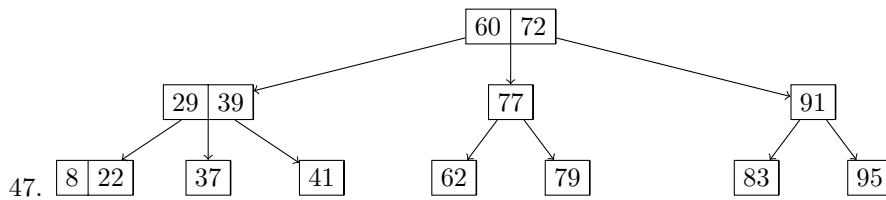
- |         |         |
|---------|---------|
| 1. (b)  | 21. (c) |
| 2. (c)  | 22. (d) |
| 3. (d)  | 23. (c) |
| 4. (a)  | 24. (c) |
| 5. (c)  | 25. (b) |
| 6. (b)  | 26. (c) |
| 7. (b)  | 27. (b) |
| 8. (b)  | 28. (a) |
| 9. (c)  | 29. (a) |
| 10. (b) | 30. (c) |
| 11. (a) | 31. (a) |
| 12. (d) | 32. (c) |
| 13. (a) | 33. (a) |
| 14. (b) | 34. (d) |
| 15. (a) | 35. (c) |
| 16. (d) | 36. (a) |
| 17. (c) | 37. (c) |
| 18. (d) | 38. (d) |
| 19. (b) | 39. (b) |
| 20. (c) | 40. (d) |

## II. Trả lời ngắn (30p)

41.  $O(\log_k(n))$
42. (a) `pRoot == NULL`  
(b) `return 0`  
(c) dòng 15: `1 + max(height(root->left()), height(root->right()))`
43. (a) `root->element() < left->element()` (b) `root->element() > right->element()` (c) `checkbst(root->left(), left, root)`  
&& `checkbst(root->right(), root, right)`
44. `[14], [7, 8], [5, 9], [11, 13]`.
45. `[15, 15, 10, 13, 2, 9, 5, 4]`



46.



48. NLR: 4 2 1 3 6 5 9 7

49.  $2 + 7 + 10 + 15 = 34$

50. (22, 18, 13, 17, 4, 35, 44, 64, 99, 67)

### III. Câu hỏi Harmony (10p)

51. `/*(a)*/ this->getRecr(root->left, index)`

`/*(b)*/ this->getRecr(root->right, (index - root->lLen))`

52. `/*(a)*/ NULL`

`/*(b)*/ this->root`

`/*(c)*/ otherStr.root`

53. `/*(a)*/ reverseRecr(root->right)`

`/*(b)*/ reverseRecr(root->left)`

`/*(c)*/ (newRoot->left->data != NULL) ? (newRoot->left->data->length()) : (newRoot->left->lLen + newRoot->left->rLen)`

`/*(d)*/ (newRoot->right->data != NULL) ? (newRoot->right->data->length()) : (newRoot->right->lLen + newRoot->right->rLen)`